Processos e Threads

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

Departamento de Informática
Universidade Estadual de Maringá

Introdução

- Computadores podem fazer várias tarefas ao “mesmo tempo”.
- Em um sistema **multiprogramado**, a CPU troca de programa para programa, cada um executando por alguns **milisegundos**.
- Em um certo instante, a CPU executa apenas um programa por vez.
- Num período, vários programas são executados, dando aos usuários a ilusão de paralelismo.
- Fala-se em **pseudoparalelismo**, em contraste com máquinas que tem 2 ou mais processadores.

O Modelo de Processos

- Todo software que pode executar num certo momento num computador (incluindo o SO) é organizado em um certo número de **processos sequenciais**.
- Um processo é um **programa em execução**.
- Conceitualmente, cada processo possui sua própria CPU.
- Na prática, a CPU é chaveada rapidamente entre todos os processos (**multiprogramação**).
- Pela velocidade da CPU e pelo pouco tempo que um processo fica esperando a CPU, os usuários têm a ilusão de que trabalham em paralelo (**pseudoparalelismo**).

Criação de Processos

- Existem quatro eventos principais que causam a criação de processos:
  - Inicialização do sistema;
  - Um processo em execução chama uma *system call* de criação de processos;
  - Um usuário pede para criar um novo processo;
  - Início de um *job batch*.
    - Batch é um arquivo de computador utilizado para automatizar tarefas. É um conjunto de comandos rodados sequencialmente. Podemos compará-lo aos scripts do Unix

Término de Processos

- Após a realização de seu trabalho, um processo deve terminar.
- Há <u>quatro</u> condições para o encerramento de um processo:
  - Exit normal (voluntário) – o processo encerrou sua tarefa e termina;
  - Exit por erro (voluntário) – o processo descobre que não tem condições para continuar executando;
  - Erro fatal (involuntário) – em geral devido a um **bug**, o processo comete um erro de execução e é encerrado;
    - Em alguns casos, é possível informar que se quer tratar alguns desses erros. Nesse caso, o processo é interrompido e desviado para uma rotina de tratamento.
  - Morto por outro processo (involuntário) – quem mata deve ter autoridade para fazer isso (no Unix, usa-se a função **kill,** no Windows o **Task Manager**).

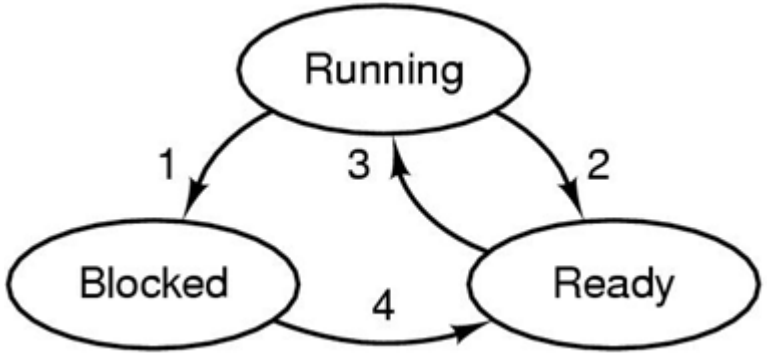
Hierarquia de Processos

- Em alguns sistemas, um processo pode gerar outros. Nesse caso, o 1º é o **PAI** e os demais são **FILHOS**.
- No Unix, um pai e todos os seus filhos formam um **grupo de processos**.
- Processo de inicialização do Unix:
  - No boot, um processo especial chamado **init** é disparado;
  - Quando ele inicia, ele lê um arquivo que diz quantos terminais existem;
  - Para cada um, é feito um **fork** de um novo processo (**login**);
  - Se alguém fizer um login OK, o programa login executa um **shell** naquele terminal;
  - O usuário pode disparar novos processos através do shell.
- No Windows, não há o conceito de hierarquia. Cada processo disparado é independente de seu pai.

Estados de Processos

- Existem 3 estados possíveis para um processo:
  - **Executando**: Usando a CPU neste instante;
  - **Pronto**: Pode executar, mas não há CPU disponível;
  - **Bloqueado**: Incapaz de executar pela espera de algum evento externo.

1. O processo é bloqueado para esperar uma entrada de dados.
2. O escalonador escolhe outro processo.
3. O escalonador escolhe este processo.
4. A entrada de dados é realizada.



Implementação de Processos:
Tabela de Processos

- Exemplo de Informações mais importantes presentes em uma tabela de Processos:

Gerenciamento de processos

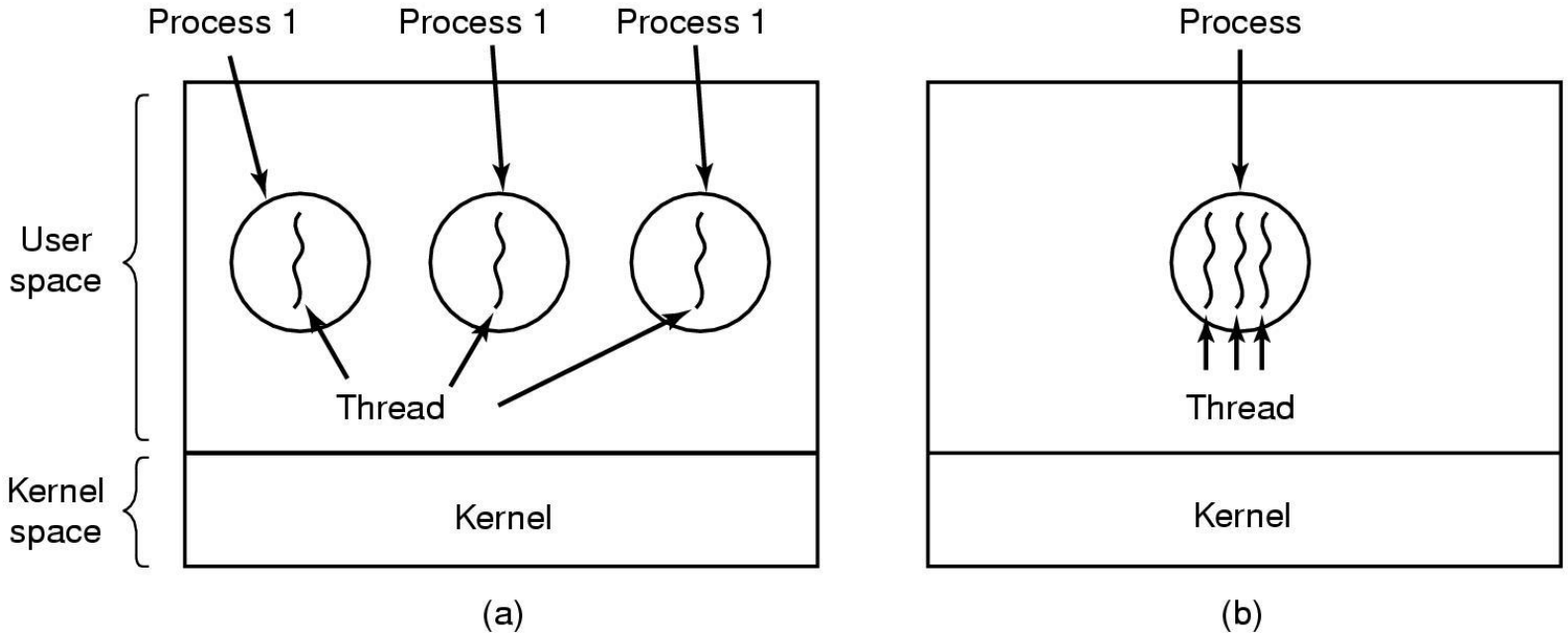
Registradores
Contador do programa
Palavra de status do programa
Ponteiro de pilha
Estado do processo
Tempo em que o processo iniciou
Tempo de CPU utilizado
Tempo de CPU dos filhos
Tempo do próximo alarme
Ponteiros de fila de mensagem
Bits de sinal pendente
Id do processo
Vários bits de sinalização

Threads

- Cada processo possui seu contador de instruções. O caminho dentro do programa que o contador faz é chamado de ***thread of execution***, ou simplesmente ***thread***.
- Associado com uma thread, temos registradores (mantém os itens de trabalho atuais) e pilha (mantém o histórico de chamadas a procedimentos).
- Quando múltiplas threads estão presentes no mesmo espaço de endereço, alguns campos da Tabela anterior não são por processo, mas por Thread, então uma tabela separada de Thread é necessária, com uma entrada por Thread.
  - Contador de Programa: Threads podem ser suspensas e retomadas;
  - Registradores: Se as Threads são suspensas, seus registradores devem ser salvos;
  - Estado: Também podem estar nos estados *em execução, pronto ou bloqueado*.

Threads

- <u>Diferença</u> entre processo e thread:
  - Processos são conjuntos de recursos agrupados para melhor gerenciamento;
  - Threads são unidades de execução organizadas para usar a CPU.

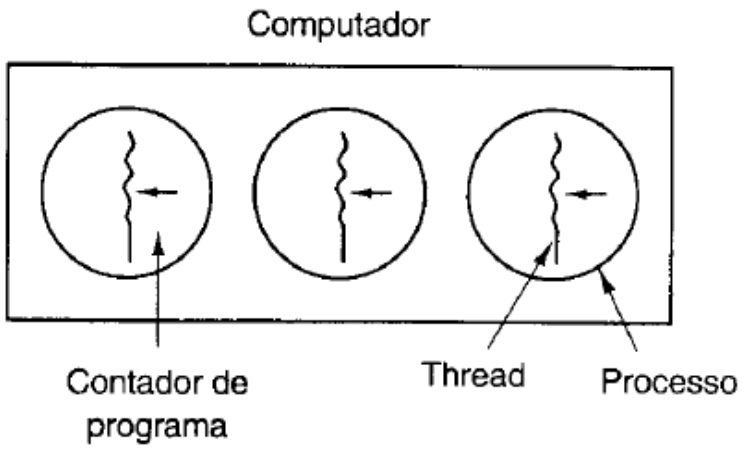


Threads

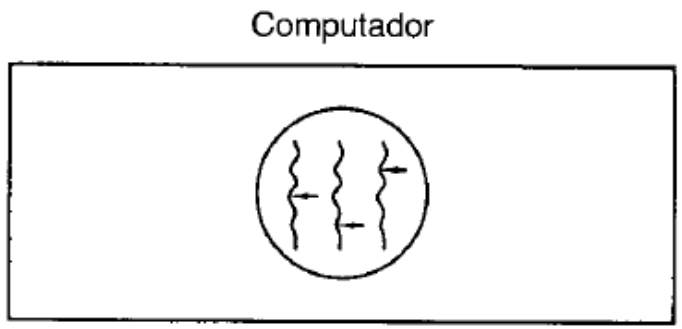
- O conceito de threads acrescenta a capacidade de múltiplas linhas de execução para o mesmo processo. Até certo grau, essas linhas são independentes uma das outras.
- Até certo grau, um processo com múltiplas threads é similar a um computador com múltiplos processos.
- As threads de um processo compartilham seu espaço de endereçamento, arquivos abertos e outros recursos.
- Os processos de uma máquina compartilham memória, discos, impressoras, etc.
- Threads possuem algumas características de processos, por isso são chamadas de ***lightweight processes***.

Threads

- Na Figura (a) temos três processos “tradicionais” cada um com seu próprio espaço de endereço e uma única linha de controle. Já na Figura (b), vemos um único processo com três linhas de controle.
- Em ambos temos três threads, mas na Figura (a) cada um possui seu espaço de endereço, enquanto que na Figura (b) as três compartilham o mesmo espaço de endereço.



(a)



(b)

Threads

- Como as threads compartilham o mesmo espaço de endereçamento do processo, possuem as mesmas variáveis globais, podem acessar itens de outras threads (incluindo stack). Assim, uma thread pode gravar sobre partes vitais de outras.

- Não há proteção entre threads porque:
  - É impossível!
  - Não deveria ser necessário!

Threads

- Processos diferentes podem pertencer a usuários diferentes. Assim, são naturalmente “concorrentes”.
- Já várias threads do mesmo processo pertencem ao mesmo usuário, que criou-as justamente para que cooperem, e não “briguem” entre si!
- Uma thread pode estar em dos possíveis estados:
  - Executando,
  - Bloqueada,
  - Pronta, ou
  - Terminada.

Threads

- O mecanismo de funcionamento das threads é baseado em uma série de system calls:
  - **Thread_create**: permite criar novas threads indicando uma procedure qualquer;
  - **Thread_exit**: a thread encerra e desaparece do sistema – não pode mais ser escalonada;
  - **Thread_wait**: permite que uma thread espere pelo resultado de outra;
  - **Thread_yield**: permite a uma thread liberar a CPU para as demais. Isso é interessante porque não há ***timesharing*** entre threads.
    - Portanto, se uma delas verifica que está gastando muita CPU é válido permitir que as demais também usem o processador.

Uso de Threads

- Múltiplas atividades podem ser realizadas dentro de uma aplicação. Algumas delas podem exigir o bloqueio do processo.
- Esse pode ser o argumento para se ter múltiplos processos para uma aplicação.
- Entretanto, pode-se necessitar de compartilhamento do mesmo espaço de endereçamento.
- Isso explica porque múltiplos processos podem não funcionar.
- Threads são fáceis de criar e destruir, já que não possuem recursos associados.
- Em alguns sistemas, criar uma thread é 100 vezes mais rápido que criar um processo.

Uso de Threads

- Threads permitem ganhos de desempenho quando realizam E/S. Uma thread pode bloquear enquanto aguarda um E/S. Nesse tempo, a CPU pode ser usada por outra.
- Threads podem ser muito úteis em sistemas com múltiplas CPUs, desde que o SO permita paralelismo entre as threads de um mesmo processo.

Uso de Threads

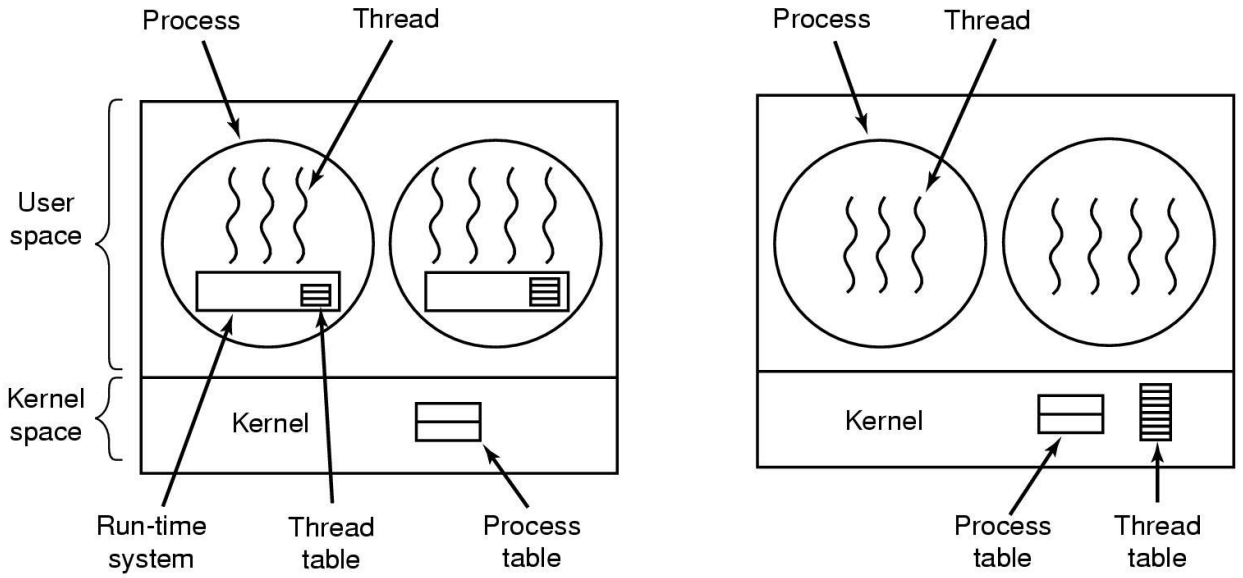
- **Exemplo:** Considere uma pessoa que está escrevendo um livro de cerca de 1000 páginas. Ela executa um comando para salvar o livro.
- Com uma única thread o processo ficaria bloqueado para comandos de teclado e mouse enquanto realiza o I/O necessário para salvar o que foi feito até o momento.
- Com múltiplas threads, é possível salvar e ainda aceitar novos comandos.
- Nesse caso, múltiplos processos não funcionam porque é necessário operar sobre o mesmo documento.

Implementando Threads

- Existem dois modos de implementar threads:
  - No espaço do usuário;
  - No kernel.

Implementando Threads:
No espaço do Usuário

- No espaço do usuário, o kernel não sabe nada sobre threads. Do ponto de vista do kernel, ele manipula processos com uma só thread.
- Uma vantagem desse método é que um pacote de threads no espaço do usuário pode ser implementado em SOs que não reconhece threads.



Implementando Threads:
No espaço do Usuário

- Cada processo mantém um sistema *run-time* para suas threads e sua própria tabela de threads.
- Quando uma thread faz algo que causa o seu bloqueio, os registradores devem ser salvos na tabela de threads e o processamento é entregue a outra thread.
- Esse procedimento é mais rápido que fazer um *trap* para o kernel. O escalonamento de threads é muito mais rápido.
- Cada processo pode ter seu próprio algoritmo de escalonamento.
- Mas não há apenas vantagens.
- Se as *system calls* são **bloqueantes**, chamar uma delas numa thread significa bloquear todas.

Implementando Threads:
No espaço do Usuário

- O argumento fundamental contra threads no espaço do usuário é que os programadores querem threads em geral para fazer chamadas bloqueantes. Nesse caso, não há vantagens nesse método.
- Não é interessante usar threads em aplicações CPU *Bound* (Ex.: Cálculos Científicos, Jogar Xadrez, etc.).
- **Observação:** CPU *Bound* é quando o tempo de processamento depende mais do processador do que das entradas e saídas, fazendo assim com que atrapalhe o tempo total de processamento.

Implementando Threads:
No Kernel

- Se o kernel sabe da existência das threads, não é necessário *run-time* nos processos.
  - Não é preciso tabelas de threads por processo; o kernel mantém uma única para todas as threads do sistema.
- Todas as chamadas bloqueantes são implementadas como **system calls**, com um custo bastante alto.
  - Porém, o bloqueio de uma thread permite passar o processador para outra thread do mesmo processo, ou de outro processo.

Implementando Threads:
No Kernel

- Devido ao custo mais alto de criação e destruição de threads, alguns S.O.s usam um sistema de reciclagem:
  - As threads destruídas são apenas marcadas como não *runnable*;
  - Se uma nova thread precisa ser criada, uma das velhas é reativada;
  - Reciclagem pode ser feita em threads a nível de usuário, mas como o custo de criação é bem menor o ganho não é tão interessante.

Dúvidas?